UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Lab Manual

Name of the Manual : Windows Log Analysis with PowerShell

Manual No : 01

Course Title : Threat Modelling and Security Monitoring Sessional

Course Code : SEC 204

Submitted By:	Submitted To:
Name : Sabeda Afrin Student ID : 2404018 Level : 2 Term : 1 Department of Cyber Security Engineering, UFTB	Name : Md Masud Rana Designation : Lecturer Department of Cyber Security Engineering, UFTB

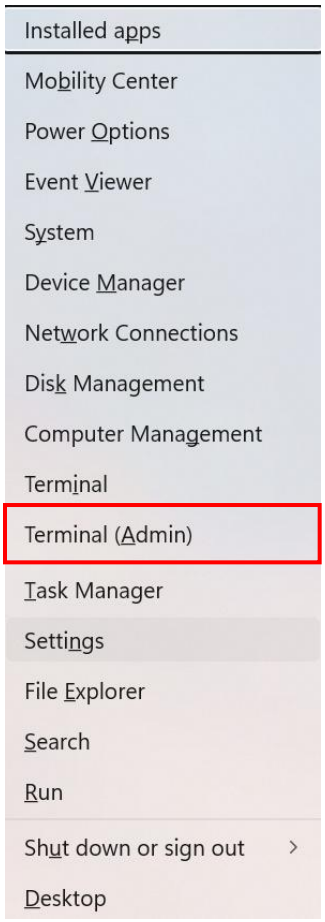
Objective

Use PowerShell to analyze Windows Security Event Logs and detect security incidents such as brute-force attacks, lateral movement, and privilege escalation.

PowerShell Event Log Queries

Step: Open PowerShell as Administrator

1. Press Win + X
2. Select Terminal (Admin).
3. Verify the version.



Basic PowerShell Event Log Queries

Detecting the powershell version:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> $PSVersionTable.PSVersion

Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      26100  9168
```

Exercise 1: Visiting Recent Security Events

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WinEvent -LogName Security -MaxEvents 10

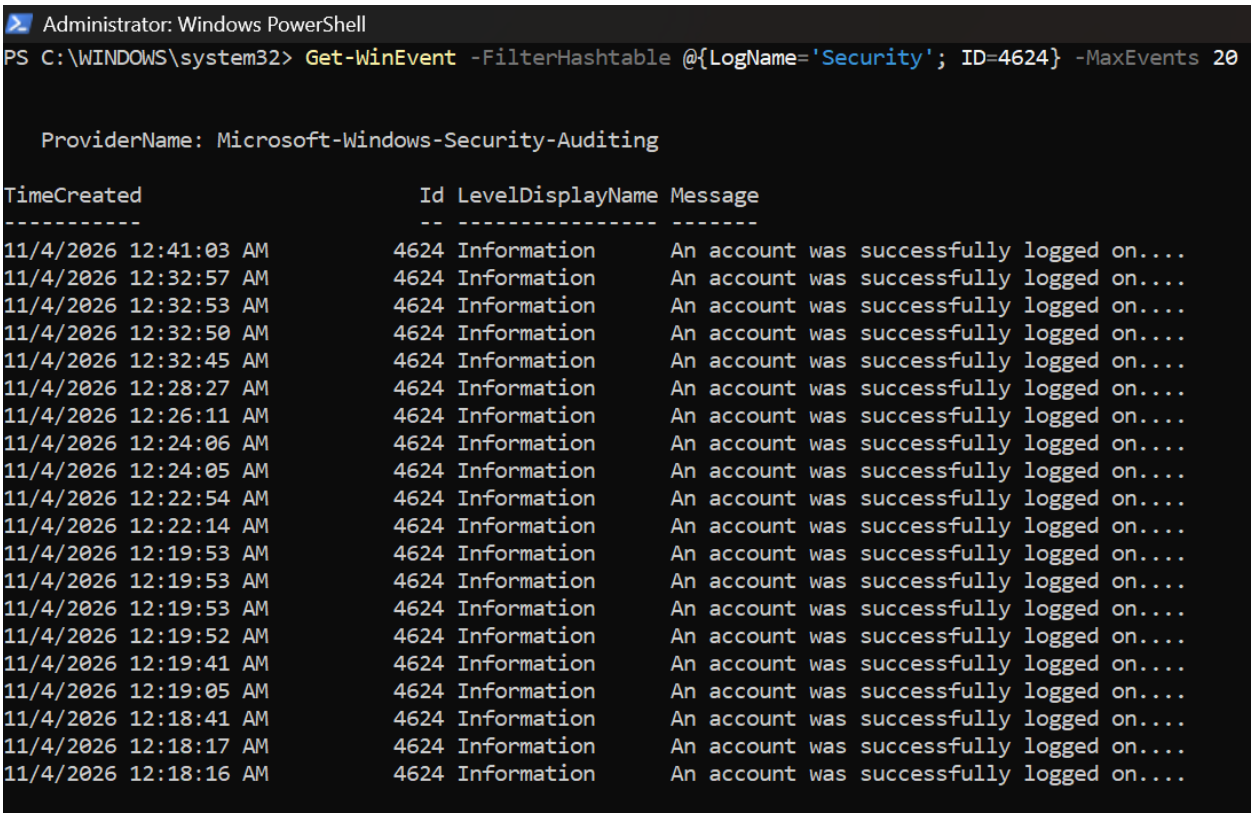

   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- --------------- -------
11/4/2026 12:32:57 AM         4672 Information     Special privileges assigned to new logon....
11/4/2026 12:32:57 AM         4624 Information     An account was successfully logged on....
11/4/2026 12:32:53 AM         4672 Information     Special privileges assigned to new logon....
11/4/2026 12:32:53 AM         4624 Information     An account was successfully logged on....
11/4/2026 12:32:50 AM         4672 Information     Special privileges assigned to new logon....
11/4/2026 12:32:50 AM         4624 Information     An account was successfully logged on....
11/4/2026 12:32:45 AM         4672 Information     Special privileges assigned to new logon....
11/4/2026 12:32:45 AM         4624 Information     An account was successfully logged on....
11/4/2026 12:30:26 AM         5379 Information     Credential Manager credentials were read....
11/4/2026 12:30:25 AM         5379 Information     Credential Manager credentials were read....
```

- **Get-WinEvent**: Used to collect event information stored in Windows, helping users monitor system activities and investigate security issues.
- **-LogName Security**: Tells PowerShell to look specifically at the Security log, where Windows records events such as login attempts and user account activities.
- **-MaxEvents 10**: Displays only the latest 10 records from the selected log instead of showing all available events.

Exercise 2: Filter by Event ID

View the last 20 successful logons (4624):



Explaination:

- **-FilterHashtable**: Used to filter event logs based on specific conditions, making it easier and faster to find relevant events.
- **@{LogName='Security'; ID=4624}**: Specifies the filtering criteria. Here, LogName='Security' selects the Security log, and ID=4624 filters events with Event ID 4624, which represents a successful logon.

Exercise 3: Filter by Time Range

```
PS C:\WINDOWS\system32> $StartTime = Get-Date "2026-11-03 00:00:00"
PS C:\WINDOWS\system32> $EndTime = Get-Date "2026-11-04 00:00:00"
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>> Logname='Security'
>> Starttime=$StartTime
>> EndTime=$EndTime
>> }


   ProviderName: Microsoft-Windows-Eventlog

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
11/3/2026 6:33:30 PM          1100 Information      The event logging service has shut down.


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
11/3/2026 6:33:28 PM          4672 Information      Special privileges assigned to new logon....
11/3/2026 6:33:28 PM          4624 Information      An account was successfully logged on....
11/3/2026 6:33:28 PM          4647 Information      User initiated logoff:...
11/3/2026 6:30:34 PM          5379 Information      Credential Manager credentials were read....
11/3/2026 6:30:34 PM          5379 Information      Credential Manager credentials were read....
11/3/2026 6:30:34 PM          5379 Information      Credential Manager credentials were read....
11/3/2026 6:30:34 PM          5379 Information      Credential Manager credentials were read....
11/3/2026 6:30:34 PM          5379 Information      Credential Manager credentials were read....
11/3/2026 6:30:34 PM          5379 Information      Credential Manager credentials were read....
11/3/2026 6:30:34 PM          4672 Information      Special privileges assigned to new logon....
11/3/2026 6:30:34 PM          4624 Information      An account was successfully logged on....
11/3/2026 6:29:31 PM          4672 Information      Special privileges assigned to new logon....
11/3/2026 6:29:31 PM          4624 Information      An account was successfully logged on....
11/3/2026 6:27:10 PM          4672 Information      Special privileges assigned to new logon....
11/3/2026 6:27:10 PM          4624 Information      An account was successfully logged on....
11/3/2026 6:23:07 PM          4672 Information      Special privileges assigned to new logon....
11/3/2026 6:23:07 PM          4624 Information      An account was successfully logged on....
```

Explanation:

- (Get-Date).AddDays(-1) = 24 hours ago
- StartTime=$StartTime = filter from that time forward

Exercise 4: Count Events

```
PS C:\WINDOWS\system32> $StartTime = (Get-Date).AddDays(-1)
PS C:\WINDOWS\system32> $FailedLogons = @(Get-WinEvent -FilterHashtable @{
>> Logname='Security'
>> ID=4625
>> StartTime=$StartTime
>> } -ErrorAction SilentlyContinue)
PS C:\WINDOWS\system32> $FailedLogons.Count
0
PS C:\WINDOWS\system32>
```

Explanation:

- Get-WinEvent – Retrieves event logs from Windows.
- -FilterHashtable – Filters event logs using specific conditions.
- LogName='Security' – Searches in the Windows Security log.
- ID=4625 – Finds failed logon attempts.
- StartTime=$StartTime – Searches events from the last 24 hours.
- $FailedLogons – Stores all the matching failed logon events.

Extracting Event Properties

Exercise 5: Understanding Event Structure

Windows events are stored as XML.

```
PS C:\WINDOWS\system32> $Event | Format-List *
PS C:\WINDOWS\system32> $Event = Get-WinEvent -FilterHashtable @{LogName='Security'
; ID=4624} -MaxEvents 1
PS C:\WINDOWS\system32> $Event | Format-List *


Message              : An account was successfully logged on.

                       Subject:
                        Security ID:           S-1-5-18
                        Account Name:          UWU$
                        Account Domain:        WORKGROUP
                        Logon ID:              0x3E7

                       Logon Information:
                        Logon Type:            5
                        Restricted Admin Mode: -
                        Remote Credential Guard:       -
                        Virtual Account:               No
                        Elevated Token:        Yes

                       Impersonation Level:           Impersonation

                       New Logon:
                        Security ID:           S-1-5-18
                        Account Name:          SYSTEM
                        Account Domain:        NT AUTHORITY
                        Logon ID:              0x3E7
                        Linked Logon ID:               0x0
                        Network Account Name:  -
                        Network Account Domain: -
                        Logon GUID:            {00000000-0000-0000-0000-00000000000
0}

                       Process Information:
                        Process ID:            0x654
                        Process Name:          C:\Windows\System32\services.exe
```

```
                      key. This will be 0 if no session key was requested.
Id                   : 4624
Version              : 3
Qualifiers           :
Level                : 0
Task                 : 12544
Opcode               : 0
Keywords             : -9214364837600034816
RecordId             : 361571
ProviderName         : Microsoft-Windows-Security-Auditing
ProviderId           : 54849625-5478-4994-a5ba-3e3b0328c30d
LogName              : Security
ProcessId            : 1648
ThreadId             : 1732
MachineName          : Uwu
UserId               :
TimeCreated          : 11/4/2026 1:19:20 AM
ActivityId           : 98349a75-6bbf-0008-ed9a-3498bf6bdd01
RelatedActivityId    :
ContainerLog         : Security
MatchedQueryIds      : {0}
Bookmark             : System.Diagnostics.Eventing.Reader.EventBookmark
LevelDisplayName     : Information
OpcodeDisplayName    : Info
TaskDisplayName      : Logon
KeywordsDisplayNames : {Audit Success}
Properties           : {System.Diagnostics.Eventing.Reader.EventProperty,
                       System.Diagnostics.Eventing.Reader.EventProperty,
                       System.Diagnostics.Eventing.Reader.EventProperty,
                       System.Diagnostics.Eventing.Reader.EventProperty...}
```

Explanation:

- $Event – Accesses the stored event.
- | (Pipeline) – Passes the event to the next command.
- Format-List – Displays event information in a list format.
- – Shows all available properties of the event.

Exercise 6: Extract Data from Failed Logons (4625)

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4625}
 -MaxEvents 10 |
>>     ForEach-Object {
>>         [PSCustomObject]@{
>>             TimeCreated = $_.TimeCreated
>>             TargetUser = $_.Properties[5].Value
>>             SourceIP = $_.Properties[19].Value
>>             LogonType = $_.Properties[8].Value
>>             FailureReason = $_.Properties[13].Value
>>         }
>>     } | Format-Table -AutoSize

TimeCreated            TargetUser SourceIP  LogonType FailureReason
-----------            ---------- --------  --------- -------------
11/1/2026 10:50:54 AM  whatever   127.0.0.1 %%2313    UWU
11/1/2026 10:50:44 AM  whatever   127.0.0.1 %%2313    UWU
10/26/2026 4:15:00 AM  whatever   -         %%2313    UWU
10/26/2026 1:16:50 AM  whatever   -         %%2313    UWU
10/14/2026 1:24:10 AM  whatever   -         %%2313    UWU
10/12/2026 12:16:41 PM whatever   127.0.0.1 %%2313    UWU
10/12/2026 12:15:14 PM whatever   127.0.0.1 %%2313    UWU
```

Explanation:

- ForEach-Object – Processes each event one by one.
- $_ – Represents the current event being processed.
- PSCustomObject – Creates a custom object to display selected event

Exercise 7: Top 5 Source IPs of Failed Logons

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $_.Properties[5].Value
>> } | Where-Object { $_ -and $_ -ne '-' } |
>>     Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object -First 5 Name, Count
Get-WinEvent : No events were found that match the specified selection criteria.
```

How it works:

- Get all 4625 events.
- Extract IP (index 19).
- Drop empty values.
- Group by IP.
- Sort descending.
- Take top 5.

Exercise 8: Top 5 Targeted Usernames

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $_.Properties[5].Value  # TargetUserName
>> } | Where-Object { $_ -ne '-' -and $_ -ne '' } |
>>     Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object -First 5 @{Name='Username';Expression={$_.Name}}, @{Name='Faile
d Attempts';Expression={$_.Count}} |
>>     Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
```

Explanation:

- Where-Object – Filters the extracted usernames.
- -ne – Means Not Equal.
- Removes - and empty usernames.
- Groups identical usernames together.
- Counts how many failed logon attempts belong to each username.
- Sorts usernames based on the number of failed attempts.
- -Descending – Displays the highest count
- Select-Object – Selects specific information.
- -First 5 – Displays only the top five usernames.
- Username – Displays the username.
- Failed Attempts – Displays the total failed attempts for each username.

Exercise 9: Logon Type Distribution (Failed Attempts)

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $LogonType = $_.Properties[8].Value
>>     switch ($LogonType) {
>>         2 { 'Interactive' }
>>         3 { 'Network' }
>>         4 { 'Batch' }
>>         5 { 'Service' }
>>         7 { 'Unlock' }
>>         8 { 'NetworkCleartext' }
>>         9 { 'NewCredentials' }
>>         10 { 'RemoteInteractive' }
>>         11 { 'CachedInteractive' }
>>         default { "Unknown ($LogonType)" }
>>     }
>> } | Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object @{Name='Logon Type';Expression={$_.Name}}, @{Name='Count';Expre
ssion={$_.Count}} |
>>     Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
```

Detecting Brute-Force Attacks

Exercise 10: Identify Brute-Force Patterns

A brute-force attack typically shows:

- Many failed logons from the same source IP.
- Attempts against multiple usernames.
- Attempts within a short time window.

Creating a ‘Detect-BruteForce.ps1’ file in notepad

Detect-BruteForce.ps1

File Edit View

```
# Define threshold
$FailureThreshold = 5
$TimeWindow = (Get-Date).AddHours(-1)

# Get failed logons
$FailedLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=$TimeWindow
} | ForEach-Object {
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        SourceIP = $_.Properties[19].Value
        TargetUser = $_.Properties[5].Value
        LogonType = $_.Properties[10].Value
    }
}

# Group by source IP
$SuspiciousIPs = $FailedLogons |
    Where-Object { $_.SourceIP -ne '-' } |
    Group-Object SourceIP |
    Where-Object { $_.Count -ge $FailureThreshold } |
    Sort-Object Count -Descending

# Display results
Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow

foreach ($IP in $SuspiciousIPs) {
    Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
    Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
    Write-Host "Targeted Accounts:" -ForegroundColor Yellow
    $IP.Group | Select-Object -ExpandProperty TargetUser -Unique | ForEach-Object {
        Write-Host "  - $_"
    }
    Write-Host ""
```

```
PS C:\WINDOWS\system32> notepad .\Detect-BruteForce.ps1
PS C:\WINDOWS\system32>  .\Detect-BruteForce.ps1
.\Detect-BruteForce.ps1 : File C:\WINDOWS\system32\Detect-BruteForce.ps1 cannot be
loaded because running scripts is disabled on this system. For more information,
see about_Execution_Policies at https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:2
+  .\Detect-BruteForce.ps1
+  ~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\WINDOWS\system32> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Cur
rentUser

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing
 the execution policy might expose you to the security risks described in the
about_Execution_Policies help topic at
https:/go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution
policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "N"):y
PS C:\WINDOWS\system32> .\Detect-BruteForce.ps1
PS C:\WINDOWS\system32>
```

```
PS C:\WINDOWS\system32> 1..10 | ForEach-Object {
>>     net use \\localhost\C$ /user:FakeUser WrongPassword
>>     Start-Sleep -Seconds 2
>> }
PS C:\WINDOWS\system32> 1..10 | ForEach-Object {
>>     net use \\localhost\C$ /user:whatever WrongPassword
>>     Start-Sleep -Seconds 2
>> }
System error 86 has occurred.

The specified network password is not correct.

System error 86 has occurred.

The specified network password is not correct.

System error 86 has occurred.

The specified network password is not correct.

System error 86 has occurred.

The specified network password is not correct.
```

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddMinutes(-5)
>> } | Select-Object TimeCreated, Message | Format-List


TimeCreated : 11/4/2026 6:30:57 AM
Message     : An account failed to log on.

              Subject:
                Security ID:            S-1-0-0
                Account Name:           -
                Account Domain:         -
                Logon ID:               0x0

              Logon Type:                       3

              Account For Which Logon Failed:
                Security ID:            S-1-0-0
                Account Name:           whatever
                Account Domain:         -

              Failure Information:
                Failure Reason:         Unknown user name or bad password.
                Status:                 0xC000006D
                Sub Status:             0xC000006A

              Process Information:
                Caller Process ID:      0x0
                Caller Process Name:    -

              Network Information:
                Workstation Name:       UWU
                Source Network Address: ::1
                Source Port:            52987

              Detailed Authentication Information:
                Logon Process:          NtLmSsp
                Authentication Package: NTLM
```

Detecting Lateral Movement

Exercise 12: Explicit Credential Usage (Event 4648)

Event ID 4648 indicates someone used explicit credentials (RunAs, PsExec, etc.) - common in lateral movement.

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4648
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceUser = $xml.Event.EventData.Data[1].'#text'
>>         TargetUser = $xml.Event.EventData.Data[5].'#text'
>>         TargetServer = $xml.Event.EventData.Data[8].'#text'
>>         Process = $xml.Event.EventData.Data[11].'#text'
>>     }
>> } | Format-Table -AutoSize

TimeCreated           SourceUser TargetUser                           TargetServer
-----------           ---------- ----------                           ------------
11/4/2026 6:30:57 AM  whatever   whatever                             Uwu
11/4/2026 6:30:24 AM  whatever   whatever                             Uwu
11/4/2026 6:29:06 AM  whatever   whatever                             Uwu
11/4/2026 6:28:23 AM  whatever   FakeUser                             Uwu
11/4/2026 6:27:50 AM  whatever   FakeUser                             Uwu
11/4/2026 6:25:22 AM  whatever   FakeUser                             Uwu
11/4/2026 6:24:49 AM  whatever   FakeUser                             Uwu
11/4/2026 6:10:20 AM  UWU$       whatever                             localhost
11/4/2026 3:39:32 AM  UWU$       whatever                             localhost
11/4/2026 1:08:35 AM  UWU$       whatever                             localhost
11/4/2026 12:22:54 AM UWU$       8D3E0E70-9CB4-4C7F-B485-6D382675C66A localhost
11/4/2026 12:18:00 AM UWU$       whatever                             localhost
11/4/2026 12:17:59 AM UWU$       5CB614CC-B2D1-4270-89BC-A1DD2B133F94 localhost
11/4/2026 12:17:46 AM UWU$       whatever                             localhost
11/4/2026 12:17:45 AM UWU$       DWM-1                                localhost
11/4/2026 12:17:45 AM UWU$       UMFD-1                               localhost
11/4/2026 12:17:45 AM UWU$       UMFD-0                               localhost
11/3/2026 5:36:07 PM  UWU$       35271D2A-7D85-4EA8-9B6E-2BE963B14D9D localhost
11/3/2026 5:21:04 PM  UWU$       whatever                             localhost
11/3/2026 5:14:20 PM  UWU$       whatever                             localhost
11/3/2026 5:14:19 PM  UWU$       DWM-1                                localhost
11/3/2026 5:14:19 PM  UWU$       UMFD-1                               localhost
11/3/2026 5:14:19 PM  UWU$       UMFD-0                               localhost
```

Explanation:

- $xml = [xml]$_.ToXml() – Converts the event details into XML format.
- 8. TimeCreated – Displays the date and time of the event.
- 9. SourceUser – Shows the user account that started the process.

- 10. TargetUser – Shows the user account whose credentials were used.
- 11. TargetServer – Displays the target server or computer name.
- 12. Process – Shows the process name that used the credentials.
- 13. [PSCustomObject] – Creates a custom object to organize the event information.

Exercise 13: Network Share Access (Event 5140)

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=5140
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         SourceIP = $xml.Event.EventData.Data[3].'#text'
>>         ShareName = $xml.Event.EventData.Data[4].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
```

Detecting Privilege Escalation

Exercise 14: Admin Group Changes (Event 4732)

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4732
>>     StartTime=(Get-Date).AddDays(-7)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         AddedUser = $xml.Event.EventData.Data[0].'#text'
>>         GroupName = $xml.Event.EventData.Data[2].'#text'
>>         AddedBy = $xml.Event.EventData.Data[6].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
```

Exercise 15: Special Privilege Assignments (Event 4672)

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4672
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         LogonID = $xml.Event.EventData.Data[3].'#text'
>>     }
>> } | Where-Object { $_.Account -notlike '*$' } |  # Filter out system accounts
>>     Format-Table -AutoSize

TimeCreated          Account  LogonID
-----------          -------  -------
11/4/2026 6:39:47 AM SYSTEM   0x3e7
11/4/2026 6:37:27 AM SYSTEM   0x3e7
11/4/2026 6:32:59 AM SYSTEM   0x3e7
11/4/2026 6:32:45 AM SYSTEM   0x3e7
11/4/2026 6:30:46 AM SYSTEM   0x3e7
11/4/2026 6:18:08 AM SYSTEM   0x3e7
11/4/2026 6:17:59 AM SYSTEM   0x3e7
11/4/2026 6:17:45 AM SYSTEM   0x3e7
11/4/2026 6:10:20 AM whatever 0x1269e8c
11/4/2026 6:10:14 AM SYSTEM   0x3e7
11/4/2026 6:10:14 AM SYSTEM   0x3e7
11/4/2026 6:02:59 AM SYSTEM   0x3e7
11/4/2026 6:00:01 AM SYSTEM   0x3e7
11/4/2026 5:51:02 AM SYSTEM   0x3e7
11/4/2026 5:47:59 AM SYSTEM   0x3e7
11/4/2026 5:47:45 AM SYSTEM   0x3e7
```

Complete Analysis Script

Exercise 16: Analyze-SecurityLogs.ps1

```
PS C:\WINDOWS\system32> <#
>> .SYNOPSIS
>>     Comprehensive Windows Security Log Analysis Script
>> .DESCRIPTION
>>     Analyzes Windows Security Event Logs for suspicious activity
>> .PARAMETER Hours
>>     Number of hours to analyze (default: 24)
>> .EXAMPLE
>>     .\Analyze-SecurityLogs.ps1 -Hours 24
>> #>
>>
>> param(
>>     [int]$Hours = 24
>> )
>>
>> $StartTime = (Get-Date).AddHours(-$Hours)
>>
>> Write-Host "`n=======================================" -ForegroundColor Cyan
>> Write-Host "Windows Security Log Analysis Report" -ForegroundColor Cyan
>> Write-Host "=======================================" -ForegroundColor Cyan
>> Write-Host "Analysis Period: Last $Hours hours" -ForegroundColor Yellow
>> Write-Host "Start Time: $StartTime" -ForegroundColor Yellow
>> Write-Host "=======================================`n" -ForegroundColor Cyan
>>
>> # 1. Failed Logon Analysis
>> Write-Host "[1] FAILED LOGON ANALYSIS" -ForegroundColor Green
>> Write-Host "---------------------------------" -ForegroundColor Green
>>
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
>>
>> if ($FailedLogons) {
>>     Write-Host "Total Failed Logons: $($FailedLogons.Count)" -ForegroundColor Yellow
>>
>>     Write-Host "`nTop 5 Source IPs:" -ForegroundColor Cyan
>>     $FailedLogons | ForEach-Object { $_.Properties[19].Value } |
>>         Where-Object { $_ -ne '-' } |
>>         Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
>>         Format-Table @{Name='IP Address';Expression={$_.Name}}, @{Name='Attempts';Expression={$_.Count}} -AutoSize
```

```
>>     Write-Host "Top 5 Targeted Accounts:" -ForegroundColor Cyan
>>     $FailedLogons | ForEach-Object { $_.Properties[5].Value } |
>>         Where-Object { $_ -ne '-' } |
>>         Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
>>         Format-Table @{Name='Username';Expression={$_.Name}}, @{Name='Attempts';Expression={$_.Count}} -AutoSize
>> } else {
>>     Write-Host "No failed logons found." -ForegroundColor Green
>> }
>>
>> # 2. Successful Logon Analysis
>> Write-Host "`n[2] SUCCESSFUL LOGON ANALYSIS" -ForegroundColor Green
>> Write-Host "----------------------------------" -ForegroundColor Green
>>
>> $SuccessfulLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4624
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
>>
>> if ($SuccessfulLogons) {
>>     Write-Host "Total Successful Logons: $($SuccessfulLogons.Count)" -ForegroundColor Yellow
>>
>>     Write-Host "`nLogon Type Distribution:" -ForegroundColor Cyan
>>     $SuccessfulLogons | ForEach-Object {
>>         $LogonType = $_.Properties[8].Value
>>         switch ($LogonType) {
>>             2 { 'Interactive' }
>>             3 { 'Network' }
>>             10 { 'RemoteInteractive' }
>>             default { "Type $LogonType" }
>>         }
>>     } | Group-Object | Sort-Object Count -Descending |
>>         Format-Table @{Name='Logon Type';Expression={$_.Name}}, @{Name='Count';Expression={$_.Count}} -AutoSize
>> }
>>
>> # 3. Account Management
>> Write-Host "`n[3] ACCOUNT MANAGEMENT EVENTS" -ForegroundColor Green
>> Write-Host "----------------------------------" -ForegroundColor Green
>>
>> $AccountEvents = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4720,4722,4724,4732
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
```

```
>> if ($AccountEvents) {
>>     Write-Host "Account Management Events: $($AccountEvents.Count)" -ForegroundColor Yellow
>>     $AccountEvents | Group-Object Id |
>>         Format-Table @{Name='Event ID';Expression={$_.Name}}, @{Name='Count';Expression={$_.Count}} -AutoSize
>> } else {
>>     Write-Host "No account management events found." -ForegroundColor Green
>> }
>>
>> # 4. Privilege Escalation
>> Write-Host "`n[4] PRIVILEGE ESCALATION INDICATORS" -ForegroundColor Green
>> Write-Host "----------------------------------" -ForegroundColor Green
>>
>> $PrivEsc = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4672,4648
>>     StartTime=$StartTime
>> } -ErrorAction SilentlyContinue
>>
>> if ($PrivEsc) {
>>     Write-Host "Privilege-Related Events: $($PrivEsc.Count)" -ForegroundColor Yellow
>>     $PrivEsc | Group-Object Id |
>>         Format-Table @{Name='Event ID';Expression={$_.Name}}, @{Name='Count';Expression={$_.Count}} -AutoSize
>> }
>>
>> Write-Host "`n========================================" -ForegroundColor Cyan
>> Write-Host "Analysis Complete" -ForegroundColor Cyan
>> Write-Host "========================================`n" -ForegroundColor Cyan
```

Exporting Results

Exercise 18: Export to HTML Report

```
PS C:\WINDOWS\system32> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[8].Value
>>     }
>> }
>>
>> $HTML = $FailedLogons | ConvertTo-Html -Title "Failed Logon Report" -PreContent "<h1>Failed Logon Analysis</h1><p>Generated: $(Get-Date)</p>"
>>
>> $HTML | Out-File "FailedLogons.html"
>>
>> Write-Host "Exported to FailedLogons.html"
Exported to FailedLogons.html
```

Conclusion

In this lab I have used PowerShell to query, filter, and parse Windows Security Event Logs, detect brute-force attacks through repeated 4625 events, spot lateral movement through 4648 and 5140, track privilege escalation through 4732 and 4672, and automate the process into reusable scripts with CSV/HTML reporting.